

Атомарность

Атомарные операции — это неделимые операции, которые выполняются целиком, без возможности прерывания их выполнения другими потоками или процессами. Пока один поток выполняет атомарную операцию, никакой другой поток не может увидеть промежуточное состояние данных или изменить их.

Пример в котором переменная `counter` увеличивается сразу из двух потоков, что неминуемо приводит к состоянию гонки:

```
#include <stdio.h>
#include <threads.h>

int counter = 0;

int thread_f(void *arg){
    for(int i = 0; i < 100000; ++i)
        counter = counter + 1;
    return 0;
}

int main(){
    thrd_t thr;
    thrd_create(&thr, thread_f, NULL);

    for(int i = 0 ; i < 100000 ; ++i)
        counter = counter + 1;

    thrd_join(thr, NULL);
    printf("counter: %d\n", counter);
    return 0;
}
```

Ассемблер, который генерирует компилятор gcc выглядит следующим образом:

...

```
movl counter(%rip), %eax // данные в регистр eax
```

```
addl $1, %eax // инкремент значения в регистре eax
```

```
movl %eax, counter(%rip) // возврат значения из eax
```

...

Операция инкремент выполняется за три действия. Между этими действиями может вмешаться другой поток.

Большинство современных архитектур процессоров, такие как Intel x86 и x86_64 имеют инструкции, позволяющие выполнить определенные инструкции атомарно, то есть за одну инструкцию, так, что никто не может вмешаться в выполнение извне. На практике, для архитектуры Intel x86_64 возможно выполнить атомарно операции с целыми числами длиной 1, 2, 4, 8 и 16 байт.

Атомарные операции можно разделить на три вида:

- 1) Атомарные операции чтения (load) и записи (store).
- 2) Атомарные операции сравнения и обмена (compare-and-swap, test-and-set, etc.).
- 3) Атомарные операции чтения, модификации и записи (read-modify-write).

Пример атомарных операций типа `read-modify-write`, реализованных встроенными в компилятор функциями. В отличие от обычных встраиваемых (`intrinsics`) в компилятор функций они не начинаются с префикса `"__builtin_"` (например `__builtin_prefetch`, `__builtin_popcount` и тд).

Эти функции можно использовать с различными компиляторами, например GCC и clang.

Атомарные операции, возвращающие значение переменной перед изменением:

```
type __sync_fetch_and_add (type *ptr, type value);
type __sync_fetch_and_sub (type *ptr, type value);
type __sync_fetch_and_or (type *ptr, type value);
type __sync_fetch_and_and (type *ptr, type value);
type __sync_fetch_and_xor (type *ptr, type value);
type __sync_fetch_and_nand (type *ptr, type value);
```

Атомарные операции, возвращающие значение переменной после изменения:

```
type __sync_add_and_fetch (type *ptr, type value);
type __sync_sub_and_fetch (type *ptr, type value);
type __sync_or_and_fetch (type *ptr, type value);
type __sync_and_and_fetch (type *ptr, type value);
type __sync_xor_and_fetch (type *ptr, type value);
type __sync_nand_and_fetch (type *ptr, type value);
```

В примере используется атомарная операция `__sync_add_and_fetch`:

```
#include <stdio.h>
#include <threads.h>

volatile int counter = 0;

int thread_f(void *arg){
    for(int i = 0; i < 100000; ++i)
        __sync_add_and_fetch(&counter, 1);

    return 0;
}

int main(){
    thrd_t thr;
    thrd_create(&thr, thread_f, NULL);

    for(int i = 0 ; i < 100000 ; ++i)
        __sync_add_and_fetch(&counter, 1);

    thrd_join(thr, NULL);
    printf("counter: %d\n", counter);
    return 0;
}
```

При генерации ассемблера для x86-64 можно убедиться, что инкремент теперь работает в одну инструкцию с дополнительным флагом `lock`:

...

```
lock addl $1, counter(%rip)
```

...

Встроенные функции для совершения атомарных операции сравнения и обмена. Функция `__sync_XXXX_compare_and_swap` обменивает значения `*ptr` и `newval` в том случае, если `*ptr == oldval`.

Версия функции, которая возвращает `bool` возвращает `true` в случае если сравнение прошло успешно.

```
bool __sync_bool_compare_and_swap  
(type *ptr, type oldval, type newval);
```

```
type __sync_val_compare_and_swap  
(type *ptr, type oldval, type newval);
```

Функция `__sync_lock_test_and_set`, обменивает атомарно значения без условия. Возвращает предыдущее значение `*ptr`:

```
type __sync_lock_test_and_set (type* ptr, type  
val);
```

Атомарные типы в C11

В стандарте C11 введены атомарные операции, которые предоставляются через заголовочный файл `stdatomic.h`. Они предназначены для реализации безопасных операций с общими данными в многопоточных программах в рамках стандарта. Убедиться в возможности использования атомарных операций можно по наличию макроса `__STDC_NO_ATOMICS__`.

Для указания того, что переменная может иметь атомарные операции используется квалификатор типа `_Atomic`. Инициализация возможна с помощью макроса `ATOMIC_VAR_INIT` или функции `atomic_init`. Например:

```
_Atomic long counter = ATOMIC_VAR_INIT(0L);
```

или

```
_Atomic long counter;  
// ...  
atomic_init(&counter, 0L);
```

Так же заголовочный файл `stdatomic.h` определяет сокращения для атомарных типов, соответствующих всем целочисленным типам. Например, тип `atomic_uchar` эквивалентен `_Atomic unsigned char`. Инициализация атомарного объекта с помощью макроса `ATOMIC_VAR_INIT` или с помощью обобщенной функции `atomic_init()`, не является атомарной операцией. Операторы инкремента и декремента `++` и `--` при применении к атомарному объекту являются атомарными. Составные операторы присваивания, такие как `+=`, работают атомарно, когда их левый операнд является атомарным объектом.

```
_Atomic long counter = ATOMIC_VAR_INIT(0L);  
// ...  
counter++; // атомарная операция
```

Так же возможно использовать атомарные операции помощью специальных функций, некоторые из них:

T

```
atomic_load (_Atomic(T) *object);
```

void

```
atomic_store (_Atomic(T) *object, T desired);
```

T

```
atomic_exchange (_Atomic(T) *object, T desired);
```

_Bool

```
atomic_compare_exchange_strong (_Atomic(T) *object,  
T *expected, T desired);
```

_Bool

```
atomic_compare_exchange_weak (_Atomic(T) *object, T  
*expected, T desired);
```

Разница

между `atomic_compare_exchange_weak` и `atomic_compare_exchange_strong` в том, что первые могут выдавать ложно-отрицательные результаты. Т.е. если значение равно ожидаемому, они вернут `false` и не заменят его на новое.

Причина существования `weak` операций в том, что на некоторых архитектурах `strong` относительно дороги. В большинстве случаев CAS инструкции крутятся в цикле (т. н. CAS loop), поэтому использование `weak` вместо `strong` не изменит логику, но может повысить производительность.

Функции позволяющие указать желательный memory_order:

T

```
atomic_load_explicit (_Atomic(T) *object, memory_order  
order);
```

void

```
atomic_store_explicit (_Atomic(T) *object, T desired,  
memory_order order);
```

T

```
atomic_exchange_explicit (_Atomic(T) *object, T desired,  
memory_order order);
```

_Bool

```
atomic_compare_exchange_strong_explicit (_Atomic(T) *object,  
T *expected, T desired, memory_order success, memory_order  
failure);
```

_Bool

```
atomic_compare_exchange_weak_explicit(_Atomic(T) *object, T  
*expected, T desired, memory_order success, memory_order  
failure);
```

Описанные ранее атомарные операции (описанные через встроенные функции так и в рамках стандарта) реализованы таким образом, что они не позволяют ни компилятору, ни исполняющему процессору выполнять переупорядочение близлежащих операций с памятью в рамках атомарной операции (`sequentially consistent`). В некоторых случаях такое поведение может привести к снижению производительности.

Чтобы смягчить это поведение, каждая атомарная операция имеет версию `_explicit()`, которая позволяет настроить переупорядочение.

Параметр типа `memory_order` может иметь одно из следующих значений.

`memory_order_relaxed` — отношение порядка между операциями с памятью не определено.

`memory_order_consume` — уровень указывает, что чтение зависит от предыдущей записи в переменные, имеющие зависимости от указанного атомарного операнда.

`memory_order_acquire` — гарантирует, что все операции чтения, выполняемые до этой атомарной операции, завершатся до начала выполнения текущей атомарной операции.

`memory_order_release` — гарантирует, что все операции записи, выполняемые после этой атомарной операции, будут видны для других потоков после завершения текущей атомарной операции записи.

`memory_order_acq_rel` — этот уровень сочетает в себе гарантии и `memory_order_acquire` и `memory_order_release`.

`memory_order_seq_cst` — Запрос последовательной согласованности включает в себя операции выборки и освобождения `memory_order_acq_rel`. Кроме того, он указывает, что все квалифицированные таким образом операции выполняются в абсолютном порядке (`sequentially consistent`), который соответствует порядку модификации участвующих в операциях атомарных объектов.

Все функции без идентификатора `_explicit()` аналогичны по эффекту оказываемому флагом `memory_order_seq_cst`.

Последовательный порядок (sequential consistency) является самым «сильным» из всех, включающим в себя все ограничения других порядков. Последовательные операции не могут быть переупорядочены: все обращения в одном потоке, происходящие до и после доступа `memory_order_seq_cst`, остаются до и после него.

Порядки `memory_order_consume` и `memory_order_acquire` могут использоваться только для операций, которые выполняют чтение.

Порядок `memory_order_release` используется только для операций, которые выполняют запись.

Порядок `memory_order_acq_rel` используется для операций, которые выполняют операцию чтения-изменения-записи (RMW).

RELAXED: Наименьшие гарантии даёт `__ATOMIC_RELAXED`:

```
x = 10;          // обычная запись
__atomic_store_n(&y, 20, __ATOMIC_RELAXED);
```

// Возможный наблюдаемый порядок в другом потоке:

// y = 20 видится до x = 10

CONSUME: Зависимости по данным гарантируется с помощью `__ATOMIC_CONSUME`:

Операции, зависящие от результата атомарной загрузки, не будут переупорядочены до неё.

// Поток 1 (инициализатор)

```
struct Data* new_data = malloc(sizeof(struct Data));
new_data->value = 42;
new_data->next = NULL;
```

// Устанавливаем указатель - RELEASE гарантирует, что инициализация завершена

```
__atomic_store_n(&global_ptr, new_data, __ATOMIC_RELEASE);
```

// Поток 2 (читатель) CONSUME:

```
struct Data* local = __atomic_load_n(&global_ptr, __ATOMIC_CONSUME);
if (local != NULL) {
    // Гарантированно видим корректно инициализированную структуру
    printf("Value: %d\n", local->value);
    printf("Next: %p\n", local->next);
}
```

// Поток 2 (читатель) RELAXED - МОЖЕТ быть проблема:

```
struct Data* local = __atomic_load_n(&global_ptr, __ATOMIC_RELAXED);
if (local != NULL) { // На слабых архитектурах МОГУТ быть видны старые/частичные данные!
    printf("Value: %d\n", local->value); // Возможно получить мусорные данные
    printf("Next: %p\n", local->next);   // Возможно получить мусорные данные
}
```

ACQUIRE: Все операции после этой атомарной операции не могут быть выполнены раньше неё. Эффект похож на **CONSUME**, однако выполняется не только для зависимых данных.

RELEASE: Все операции до атомарной операции не будут переупорядочены после неё.

ACQ_REL: Комбинация ACQUIRE и RELEASE.

SEQ_CST: Sequential Consistency даёт самые строгие гарантии — все операции видны в одном и том же порядке всем потокам.

Атомарные операции с явным указанием memory order из стандарта C11 также могут быть выражены через более новые встраиваемые в компилятор функции:

```
type __atomic_load(type *ptr, type *ret, int memorder);
void __atomic_store (type *ptr, type *val, int memorder);
type __atomic_add_fetch (type *ptr, type val, int memorder);
type __atomic_sub_fetch (type *ptr, type val, int memorder);
type __atomic_and_fetch (type *ptr, type val, int memorder);
type __atomic_xor_fetch (type *ptr, type val, int memorder);
type __atomic_or_fetch (type *ptr, type val, int memorder);
type __atomic_fetch_add (type *ptr, type val, int memorder);
type __atomic_fetch_sub (type *ptr, type val, int memorder);
type __atomic_fetch_and (type *ptr, type val, int memorder);
type __atomic_fetch_xor (type *ptr, type val, int memorder);
void __atomic_exchange (type *ptr, type *val, type *ret,
int memorder);
...
```

В их случае последним параметром может выступать:

```
__ATOMIC_RELAXED
__ATOMIC_CONSUME
__ATOMIC_ACQUIRE
__ATOMIC_RELEASE
__ATOMIC_ACQ_REL
__ATOMIC_SEQ_CST
```

Еще несколько встраиваемых функций:

```
type __atomic_load_n (type *ptr, int memorder)
void __atomic_load (type *ptr, type *ret, int memorder)
void __atomic_store_n (type *ptr, type val, int memorder)
void __atomic_store (type *ptr, type *val, int memorder)
type __atomic_exchange_n (type *ptr, type val, int memorder)
void __atomic_exchange (type *ptr, type *val, type *ret,
int memorder)
bool __atomic_compare_exchange_n (type *ptr, type *expected,
type desired, bool weak, int success_memorder,
int failure_memorder)
bool __atomic_compare_exchange (type *ptr, type *expected, ty
pe *desired, bool weak, int success_memorder,
int failure_memorder)
bool __atomic_test_and_set (void *ptr, int memorder)
void __atomic_clear (bool *ptr, int memorder)
void __atomic_thread_fence (int memorder)
void __atomic_signal_fence (int memorder)
bool __atomic_always_lock_free (size_t size, void *ptr)
bool __atomic_is_lock_free (size_t size, void *ptr)
```

(Подробнее: https://gcc.gnu.org/onlinedocs/gcc/_005f_005fatomic-Builtins.html)

Spinlock

Примитив синхронизации `spinlock` можно реализовать с помощью атомарных операций сравнения и обмена.

```
typedef struct {
    volatile int locked;
} spinlock;

void spinlock_init(spinlock* lock) {
    lock->locked = 0;
}

void spinlock_lock(spinlock* lock) {
    while(__atomic_test_and_set(&lock->locked, __ATOMIC_ACQUIRE));
}

int spinlock_trylock(spinlock* lock) {
    return !__atomic_test_and_set(&lock->locked,
__ATOMIC_ACQUIRE);
}

void spinlock_unlock(spinlock* lock) {
    __atomic_clear(&lock->locked, __ATOMIC_RELEASE);
}
```

Futex

Futex (сокращение от **fast userspace mutex**) — способ реализации базовых блокировок, примитив для построения более сложных примитивов синхронизации (семафоров, мьютексов). Фьютексы появились в ядре **Linux** в 2003 году, основной идеей было предоставить более эффективный способ синхронизации пользовательских потоков с минимальным количеством обращений к ядру ОС.

Фьютексы — это низкоуровневый инструмент, напрямую его использовать стоит лишь при разработке фундаментальных библиотек, вроде стандартной библиотеки **C/C++**.

(Описание взаимодействий с фьютексом можно найти по адресу: <https://man7.org/linux/man-pages/man2/futex.2.html> или с командой ‘**man futex**’)

Фьютекс состоит из двух частей:

- выровненной переменной 32-битного целого типа, которая размещается в `userspace`,
- очередь ожидания в `kernel space`, ассоциированная с этим целым числом.

Основная идея состоит в том, чтобы процессы и потоки могли оперировать с целым числом в пользовательском пространстве, выполняя дорогие системные вызовы только в случае, когда нужны операции с очередью ожидания (например, разбудить поток или усыпить текущий поток). Правильно спроектированный примитив синхронизации на базе `futex` при отсутствии соперничества не делает системных вызовов. Например, чтобы взять `lock`, нужно сделать лишь атомарный инкремент переменной, что относительно быстро. Если при этом появляются другие потоки, желающие захватить примитив синхронизации, то тогда выполнится системный вызов.

Сигнатура системного вызова для выполнения операций с фьютексом:

```
int syscall(int number, ...);
```

или

```
int futex(int *uaddr, int futex_op, int val, const struct timespec
*timeout, /* or: uint32_t val2 */int *uaddr2, int val3);
```

`int number` — номер системного вызова, который в данном случае равен `SYS_futex`.

`void *uaddr` — указатель на переменную, с которой происходит взаимодействие (например, мьютекс или семафор).

`int op` — операция, которую необходимо выполнить (например, `FUTEX_WAIT`, `FUTEX_WAKE` и т.д.).

`int val` — значение, используемое в операции (например, для `FUTEX_WAIT` это значение будет ожидаемым значением переменной).

`const struct timespec *timeout` — указатель на структуру `timespec`, который задает время ожидания. Этот аргумент используется только в операции `FUTEX_WAIT`.

Параметры `uaddr` и `val` позволяют судить системному вызову о возможности выполнить то или иное действие. Например при вызове `FUTEX_WAIT` поток будет заблокирован только если значение, на которое указывает `uaddr`, совпадает со значением `val`.

Для вызова `FUTEX_WAIT` аргумент `*uaddr` проверяется на равенство `val`. Если они равны, то поток засыпает. Загрузка, сравнение с ожидаемым значением и запуск сна выполняются атомарно. Если они не равны вызов немедленно завершается с ошибкой `EAGAIN`. Вызывающий всегда должен предполагать, что возвращаемое значение 0 может означать ложное пробуждение (`spurious wake-up`), и проверить значение слова фьютекса. Если аргумент `timeout` не равен `NULL`, то его содержимое определяет относительное время ожидания, измеряемое по часам `CLOCK_MONOTONIC` (этот интервал будет округлѐн до точности системных часов, и гарантируется, что он не наступит раньше положенного). Для данного вызова аргументы `uaddr2` и `val3` игнорируются. Загрузка значения слова `futex`, сравнение этого значения с ожидаемым значением и фактическая блокировка будут происходить атомарно и будут полностью упорядочены относительно параллельных операций, выполняемых другими потоками над тем же словом `futex`.

Пример вызова `FUTEX_WAIT`:

```
futex = 1;  
...  
int err = syscall(SYS_futex, (int*)&futex, FUTEX_WAIT, 1,  
0, 0, 0);
```

`FUTEX_WAKE` — пробуждает не больше `val` процессов, ожидающих слово фьютекса по адресу `uaddr`. Чаще всего, `val` присваивают или 1 (те. пробудить одного ожидающего), или `INT_MAX` (пробудить всех ожидающих). Не гарантируется, что вызов разбудит каких-то определённых ожидающих. Возвращается количество разбуженных ожидающих. Для данного вызова аргументы `timeout`, `uaddr2` и `val3` игнорируются. Выполнение `FUTEX_WAKE` также будет являться полным барьером.

Пример вызова `FUTEX_WAKE`:

```
int wc = syscall(SYS_futex, (int*)&futex,  
FUTEX_WAKE, 1 /* пробуждаем 1 поток */, 0, 0, 0);
```

В вызове `фьютекс` доступны ещё несколько флагов:

`FUTEX_PRIVATE_FLAG` — указывает ядру, что `фьютекс` доступен только для одного процесса.

`FUTEX_FD` — операция создаёт файловый дескриптор, который связан с `фьютексом`.

`FUTEX_WAKE_OP` — добавлена для работы в некоторых случаях из пользовательского пространства, в которых нужно одновременно учитывать несколько `фьютексов`, например в случае `pthread_cond_signal`.

...

Mutex

Примитив `mutex` можно описать с помощью переменной типа `int`, вызова `futex` и атомарных операций. В самом начале `mutex` разблокирован и его значение в таком случае будет 0. При попытке захвата с помощью операции `__atomic_test_and_set` значение `mutex` сравнивается с 0 и в случае успешного сравнения в `mutex` заносится значение 1. Значение 1 — `mutex` заблокирован единственной нитью. Далее, если в критическую секцию пытаются попасть другие потоки, то с помощью CAS операции в значение мютекса устанавливается 2 и нить засыпает. В случае, если несколько нитей пытаются получить доступ к защищенной секции значение `mutex` будет 2, что позволит сразу же усыпить нить, не пробегая к повторному CAS.

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/syscall.h>
#include <linux/futex.h>

volatile int mutex = 0;

void lock() {
    while (1) {
        if (__atomic_test_and_set(&mutex, __ATOMIC_ACQUIRE) == 0) {
            return;
        }

        int expected = 1;
        if (__atomic_load_n(&mutex, __ATOMIC_RELAXED) == expected) {
            syscall(SYS_futex, &mutex, FUTEX_WAIT, expected, NULL,
NULL, 0);
        }
    }
}

void unlock() {
    __atomic_store_n(&mutex, 0, __ATOMIC_RELEASE);
    syscall(SYS_futex, &mutex, FUTEX_WAKE, 1, NULL, NULL, 0);
}
```